DISASTER ALERT & MONITORING SYSTEM

Real-Time Streaming Data Pipeline & Analytics Report

Technologies Used : Apache Kafka | Apache Spark | Delta Lake | Hive Metastore | Power BI

Architecture : Producer → Bronze → Silver → Gold → BI Views

- Anshuman Sahoo
sahooo.anshuman999@gmail.com

Project Overview

The **Disaster Alert & Monitoring System** is a real-time data engineering project developed to stream, process, and analyze disaster threat signals from sensor networks deployed across disaster-prone regions. It integrates data related to sensor readings, monitoring stations, geographic regions, and alert levels to understand disaster patterns and enable rapid response.

The project uses Apache Kafka for real-time event streaming, Apache Spark Structured Streaming for data processing across **bronze, silver, and gold layers** using the Medallion Architecture, Delta Lake for reliable **lakehouse storage**, and **Power BI** for building interactive dashboards.

By analyzing threat indices, alert levels, disaster types, and critical flags, the system reveals important patterns in disaster activity that can support emergency response planning and resource allocation.

Data Pipeline Summary

The pipeline is structured into the following layers, each representing a stage of data refinement :

Layer	Description
Producer	Streams fake sensor events to Kafka topic **disaster-alert-topic** (70% clean, 30% dirty)
Bronze Layer	Ingests raw Kafka stream into Delta Lake with full schema and raw JSON preserved
Silver Layer	Applies data quality checks, type casting, deduplication, and feature engineering
Gold Layer	Produces dimension tables (**dim_region, dim_station**) and fact table (**fact_disaster**)
BI Views	Creates **bi_fact_disaster, bi_dim_region, bi_dim_station** views for Power BI consumption

• Dataset Summary

The data produced by the system is structured into the following tables in the disaster database :

Table	Type	Key Columns
fact_disaster	Fact Table	sensor_id, station_id, region, threat_index_int, alert_level, disaster_type, threat_band, critical_flag, night_flag, hour, event_ts, date
dim_region	Dimension	region, region_type, disaster_risk
dim_station	Dimension	station_id, station_type, response_time_mins

The database system in the table format can be completely fit to the Power BI Desktop . It need to converted in VIEWS format :

View	Type	Key Columns
bi_fact_disaster	BI View	All fact_disaster columns, renamed for Power BI
bi_dim_region	BI View	region, region_type, disaster_risk.
bi_dim_station	BI View	station_id, station_type, response_time_mins.

• Key Features of the Dataset

Streams real-time disaster threat readings from sensors across 5 geographic regions that are :

COASTAL, HILLSIDE, FOREST, URBAN, RIVERBANK, DESERT.

Tracks 7 disaster types :

EARTHQUAKE, FLOOD, WILDFIRE, TSUNAMI, LANDSLIDE, CYCLONE, VOLCANO.

Includes 9 dirty data types :

null values, negative indices, extreme values, duplicates, late/future events, wrong data types, schema drift, corrupt JSON to simulate real-world data quality issues.

Produces enriched features :

threat_band (LOW to CATASTROPHIC), critical_flag, night_flag for downstream analytics.

Supports real-time Power BI dashboards via Apache Spark Thrift Server using DirectQuery mode.

- ## Key Environments Setup – Creating Project Directory Launching Docker Container

1) Creating project Directory

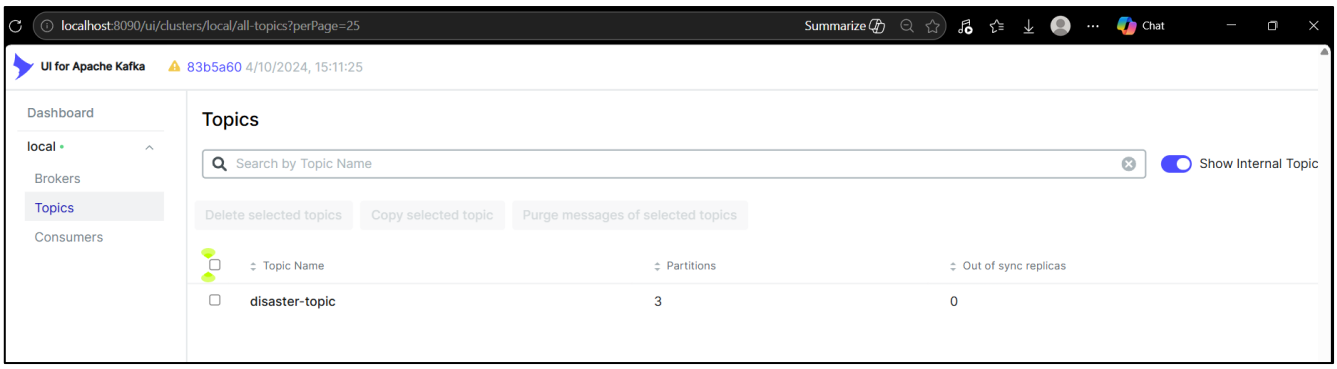
A new project directory named giit is created on the local Windows machine using **mkdir giit**, followed by navigating into it with **cd giit**.

The docker **compose up -d** command is then executed to pull and launch all required Docker images including Apache Kafka, Apache Hive, Apache Spark, Kafka-UI, and PostgreSQL. All 6 containers — **kafka, spark-master, spark-worker, hive-metastore-db, kafka-ui, and hive-metastore** — are successfully started in detached mode.

```
Windows PowerShell (x86)

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\sahoo> mkdir giit


    Directory: C:\Users\sahoo


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        27-03-2026  01:50 PM                giit


PS C:\Users\sahoo> cd giit
PS C:\Users\sahoo\giit> docker compose up -d
[+] up 69/70
 ✔Image apache/kafka:latest           Pulled                                                    1924.0s
 ✔Image apache/hive:3.1.3             Pulled                                                    2328.0s
 ✔Image apache/spark:3.5.1            Pulled                                                    2336.8s
 ✔Image provectuslabs/kafka-ui:latest Pulled                                                    1815.6s
 ✔Image postgres:13                   Pulled                                                    1820.6s
 ✔Network giit_default                Created                                                   0.3s
 ✔Container kafka                     Started                                                   3.7s
 ✔Container spark-master              Started                                                   3.8s
 ✔Container hive-metastore-db         Started                                                   3.6s
 ✔Container kafka-ui                  Started                                                   3.3s
 - Container hive-metastore           Starting                                                  3.6s
 ✔Container spark-worker              Started                                                   3.6s
Error response from daemon: failed to create task for container: failed to create shim task: OCI runtime create failed: runc create failed: unable to start
container process: error during container init: error mounting "/run/desktop/mnt/host/c/Users/sahoo/giit/hive-conf/hive-site.xml" to rootfs at "/opt/hive/co
nf/hive-site.xml": mount src=/run/desktop/mnt/host/c/Users/sahoo/giit/hive-conf/hive-site.xml, dst=/opt/hive/conf/hive-site.xml, dstFd=/proc/thread-self/fd/
11, flags=MS_BIND|MS_REC: not a directory: Are you trying to mount a directory onto a file (or vice-versa)? Check if the specified host path exists and is t
he expected type
PS C:\Users\sahoo\giit>
```

2) Building a **"TOPIC"** in kafka :

```
Windows PowerShell (x86)

      0.0s
 ✔Container spark-master      Running
      0.0s
 ✔Container kafka             Running
      0.0s
 ✔Container spark-worker      Running
      0.0s
 ✔Container kafka-ui          Running
      0.0s
 ✔Container hive-metastore    Started
      1.1s
PS C:\Users\sahoo\giit> docker exec -it kafka /opt/kafka/bin/kafka-topics.sh --create --topic disaster-topic
--bootstrap-server kafka:9092 --partitions 3 --replication-factor 1
Created topic disaster-topic.
PS C:\Users\sahoo\giit>
```



TOPIC created successfully in kafka .

3) Python Dependencies Installation — kafka-python, faker, pytz

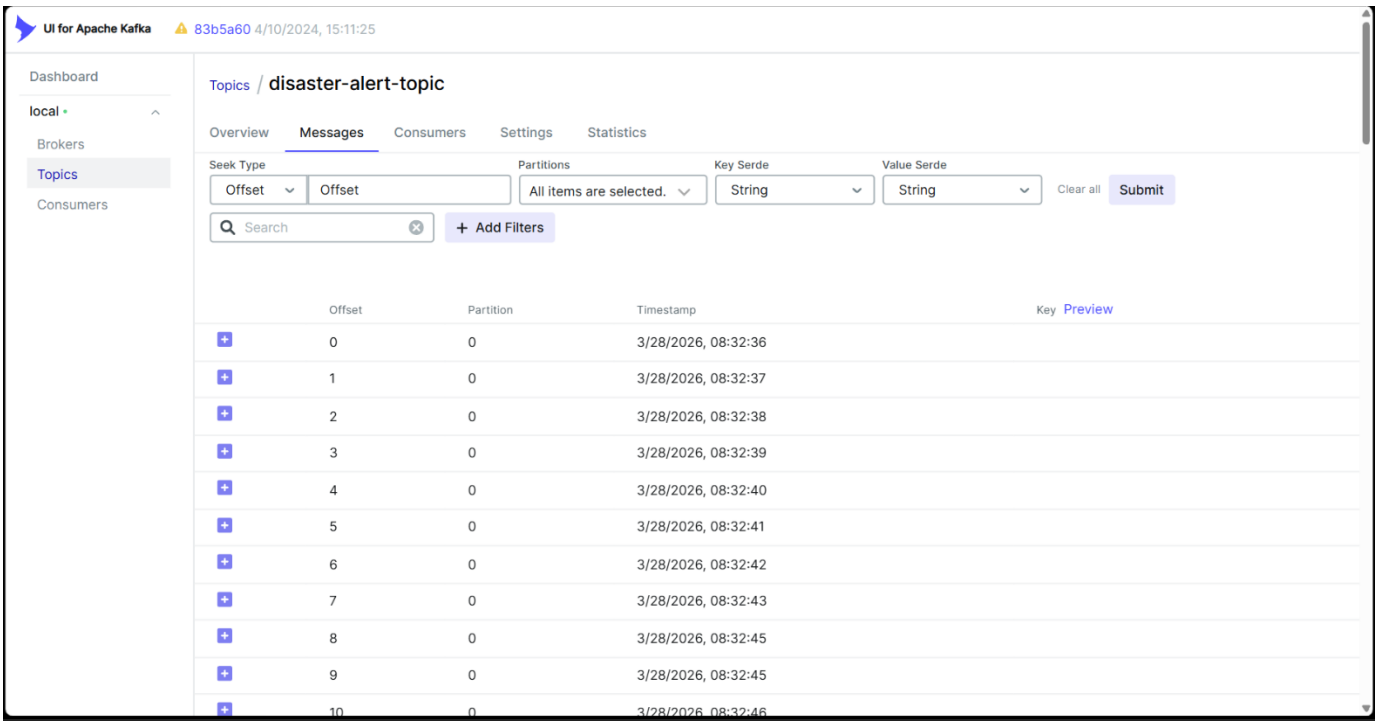
```
Windows PowerShell (x86)
PS C:\Users\sahoo\giit> pip install kafka-python faker pytz
Collecting kafka-python
  Downloading kafka_python-2.3.0-py2.py3-none-any.whl.metadata (10.0 kB)
Collecting faker
  Downloading faker-40.11.1-py3-none-any.whl.metadata (16 kB)
Collecting pytz
  Downloading pytz-2026.1.post1-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata (from faker)
  Downloading tzdata-2025.3-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading kafka_python-2.3.0-py2.py3-none-any.whl (326 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 326.3/326.3 kB 1.8 MB/s eta 0:00:00
Downloading faker-40.11.1-py3-none-any.whl (2.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 1.9 MB/s eta 0:00:00
Downloading pytz-2026.1.post1-py2.py3-none-any.whl (510 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 510.5/510.5 kB 1.3 MB/s eta 0:00:00
Downloading tzdata-2025.3-py2.py3-none-any.whl (348 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 348.5/348.5 kB 1.7 MB/s eta 0:00:00
Installing collected packages: pytz, kafka-python, tzdata, faker
  WARNING: The script faker.exe is installed in 'C:\Users\sahoo\AppData\Local\Packages\PythonSoftwareFoundati
on.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-loc
ation.
Successfully installed faker-40.11.1 kafka-python-2.3.0 pytz-2026.1.post1 tzdata-2025.3

[notice] A new release of pip is available: 24.0 -> 26.0.1
[notice] To update, run: C:\Users\sahoo\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3
.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
PS C:\Users\sahoo\giit>
```

- ## Kafka Producer – Event Simulation

The producer script simulates a network of disaster monitoring sensors continuously streaming events into the Kafka topic disaster-alert-topic.

It generates a mix of **clean (70%)** and **dirty (30%)** events to simulate real-world data quality challenges.

```
Windows PowerShell (x86)
PS C:\Users\sahoo\giit> cd producer
PS C:\Users\sahoo\giit\producer> python disaster-detector.py
 Disaster Alert Producer started - streaming to [disaster-alert-topic] ...
------------------------------------------------------------
 DIRTY | {'sensor_id': '5a0e77f2-93af-42ef-ab86-e24c32324092', 'station_id': 'DS300', 'region': 'RIVERBANK',
'threat_index': 466, 'alert_level': 1, 'disaster_type': 'WILDFIRE', 'event_time': '2026-03-28T03:02:36.476498
+00:00'}
 DIRTY | {'sensor_id': 'aea72659-d10a-47ea-aeb7-4accdc0ac11e', 'station_id': 'DS500', 'region': 'URBAN', 'thr
eat_index': 15, 'alert_level': 1, 'disaster_type': 'VOLCANO', 'event_time': '2026-03-28T01:33:37.435194+00:00
'}
 CLEAN | {'sensor_id': '0572ab10-8bf6-45d6-b83e-1afca6449fce', 'station_id': 'DS100', 'region': 'COASTAL', 't
hreat_index': 51, 'alert_level': 3, 'disaster_type': 'WILDFIRE', 'event_time': '2026-03-28T03:02:38.849744+00
:00'}
 CLEAN | {'sensor_id': '184855d4-2705-4262-8155-7e10f7baf40b', 'station_id': 'DS200', 'region': 'FOREST', 'th
reat_index': 4, 'alert_level': 1, 'disaster_type': 'VOLCANO', 'event_time': '2026-03-28T03:02:39.643900+00:00
'}
 CLEAN | {'sensor_id': 'b1ef7901-314f-4136-b448-7ed1164d5575', 'station_id': 'DS300', 'region': 'DESERT', 'th
reat_index': 80, 'alert_level': 4, 'disaster_type': 'LANDSLIDE', 'event_time': '2026-03-28T03:02:40.240453+00
:00'}
```



*result displayed in Apache Kafka.

1) Library Imports and Setup

The producer imports Kafka-Producer for streaming, Faker for generating unique sensor UUIDs, pytz for UTC timestamp handling, and standard Python libraries for randomization and time control.

The producer script begins by importing all necessary libraries — **KafkaProducer** for streaming events to Kafka, Faker for generating unique sensor **UUIDs,** and **pytz** along with datetime for accurate UTC timestamp handling.

A **Faker()** instance is initialized to generate realistic fake data, followed by a **KafkaProducer** configured to connect to the Kafka broker at **localhost:29092**

```python
from kafka import KafkaProducer
from faker import Faker
import json
import random
import time
from datetime import datetime, timedelta
import pytz

fake = Faker()

producer = KafkaProducer(
    bootstrap_servers="localhost:29092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)
```

2) Reference Data

The producer uses the following reference data to generate realistic sensor events:

Parameter	Values
stations	DS100, DS200, DS300, DS400, DS500
regions	COASTAL, HILLSIDE, FOREST, URBAN, RIVERBANK, DESERT
disaster_types	EARTHQUAKE, FLOOD, WILDFIRE, TSUNAMI, LANDSLIDE, CYCLONE, VOLCANO
threat_index range	0 – 100 (clean events)
alert_level range	1 (LOW) to 5 (CATASTROPHIC)
Kafka topic	disaster-alert-topic

3) Clean Event Generation

Clean events represent valid sensor readings. Each event contains a unique sensor_id (UUID), a randomly selected station and region, a threat_index (0–100), an auto-calculated alert_level (1–5), a disaster_type, and a UTC timestamp.

```python
# ── Dirty Event ──────────────────────────────────────────────
def generate_dirty_event():
    dirty_type = random.choice([
        "null_threat",
        "negative_threat",
        "extreme_threat",
        "duplicate_sensor",
        "late_reading",
        "future_reading",
        "threat_as_string",
        "schema_drift",
        "corrupt_json"
    ])

    base = generate_clean_event()
```

4) Dirty Event Types (9 Types)

30% of events are intentionally corrupted to simulate real-world data quality issues. The Silver layer is responsible for detecting and filtering these out.

Dirty Type	Description
null_aqi	threat_index set to None — sensor lost signal
negative_aqi	threat_index is a negative value — physically impossible
extreme_aqi	threat_index above 999 — calibration failure
duplicate_sensor	Same sensor_id reused — deduplication required
late_reading	event_time is 10–120 minutes in the past — late arrival
future_reading	event_time is 5–60 minutes ahead — clock skew
aqi_as_string	threat_index sent as 'LOW', 'HIGH' etc. — wrong type
schema_drift	Extra field aftershock_probability added unexpectedly
corrupt_json	Payload is ###CORRUPTED_ALERT### — transmission failure

• Bronze Layer – Raw Ingestion

The Bronze layer is the first processing stage. It reads raw events from the Kafka topic disaster-alert-topic and writes them as-is into a Delta Lake table.

The schema is kept flexible (**threat_index** as **StringType)** to absorb dirty events without failing.

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

# ── Spark Session Config ──────────────────────────────────────
spark = (
    SparkSession.builder
    .appName("DisasterAlertStreamingLakehouse")
    # cluster master
    .master("spark://spark-master:7077")
    # delta lake
    .config("spark.sql.extensions",
            "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
            "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .enableHiveSupport()
    .getOrCreate()
)
spark.sparkContext.setLogLevel("WARN")
```

```
PS C:\users\sahoo\giit> docker exec -it spark-worker /opt/spark/bin/spark-submit --
conf spark.jars.ivy=/tmp/.ivy --packages io.delta:delta-spark_2.12:3.2.0,org.apache
.spark:spark-sql-kafka-0-10_2.12:3.5.1 /opt/spark-apps/disaster-bronze.py
:: loading settings :: url = jar:file:/opt/spark/jars/ivy-2.5.1.jar!/org/apache/ivy
/core/settings/ivysettings.xml
Ivy Default Cache set to: /tmp/.ivy/cache
The jars for the packages stored in: /tmp/.ivy/jars
io.delta#delta-spark_2.12 added as a dependency
org.apache.spark#spark-sql-kafka-0-10_2.12 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-72b27414-4537-4ac
d-b2ef-6f4b4d490eeb;1.0
        confs: [default]
        found io.delta#delta-spark_2.12;3.2.0 in central
        found io.delta#delta-storage;3.2.0 in central
        found org.antlr#antlr4-runtime;4.9.3 in central
        found org.apache.spark#spark-sql-kafka-0-10_2.12;3.5.1 in central
        found org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.5.1 in centra
```

* Bronze layer Implementation

• Silver Layer- Data Quality & Enrichment

The Silver layer reads from the Bronze Delta table and applies comprehensive data quality checks, type casting, validation rules, deduplication, and feature engineering to produce clean, trusted data.

1) Data Quality Flag

```python
# ── Data Quality Flag ──────────────────────────────────────────
dq_df = bronze_df.withColumn(
    "dq_flag",
    when(col("sensor_id").isNull(),            "MISSING_SENSOR")
    .when(col("event_time").isNull(),          "MISSING_TIME")
    .when(col("raw_json").contains("CORRUPTED"),"CORRUPT_JSON")
    .otherwise("OK")
)
```

2) Safe Type Casting

threat_index (stored as StringType in Bronze) is safely cast to integer. Dirty string values like 'LOW' will become null and get filtered in the validation step :

```python
# ── Safe Type Casting ──────────────────────────────────────────
# threat_index was kept as StringType in bronze to absorb dirty values
# now we safely cast to int — dirty strings like "LOW" will become null
typed = dq_df.withColumn(
    "threat_index_int",
    col("threat_index").cast("int")
).withColumn(
    "event_ts",
    to_timestamp("event_time")
)
```

3) Business Validation Rules

```python
# ── Business Validation Rules ──────────────────────────────────
validated = typed.withColumn(
    "threat_valid",
    when(
        (col("threat_index_int") >= 0) & (col("threat_index_int") <= 100), 1
    ).otherwise(0)
).withColumn(
    "time_valid",
    when(
        col("event_ts") <= current_timestamp() + expr("INTERVAL 10 MINUTES"), 1
    ).otherwise(0)
)
```

4) Feature Engineering

Three new features are engineered in the **Silver layer** for downstream analytics :

Feature	Description
threat_band	Maps threat_index to LOW / MODERATE / HIGH / SEVERE / CATASTROPHIC
critical_flag	1 if threat_index >= 61 (SEVERE or above) — triggers emergency response
night_flag	1 if event occurs between 10pm–5am — disasters at night need faster response

5) Implementing Silver Transmission

```
Windows PowerShell (x86)
PS C:\Users\sahoo\giit\producer> cd ..
PS C:\Users\sahoo\giit> docker exec -it spark-worker /opt/spark/bin/spark-submit --conf spark.jars.ivy=/tmp/.
ivy --packages io.delta:delta-spark_2.12:3.2.0,org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 /opt/spark-ap
ps/disaster-bronze.py
:: loading settings :: url = jar:file:/opt/spark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings
.xml
Ivy Default Cache set to: /tmp/.ivy/cache
The jars for the packages stored in: /tmp/.ivy/jars
io.delta#delta-spark_2.12 added as a dependency
org.apache.spark#spark-sql-kafka-0-10_2.12 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-a6589f9d-ae06-4554-962c-c0e9dbd330f3;1.0
        confs: [default]
        found io.delta#delta-spark_2.12;3.2.0 in central
        found io.delta#delta-storage;3.2.0 in central
        found org.antlr#antlr4-runtime;4.9.3 in central
        found org.apache.spark#spark-sql-kafka-0-10_2.12;3.5.1 in central
        found org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.5.1 in central
        found org.apache.kafka#kafka-clients;3.4.1 in central
        found org.lz4#lz4-java;1.8.0 in central
        found org.xerial.snappy#snappy-java;1.1.10.3 in central
        found org.slf4j#slf4j-api;2.0.7 in central
        found org.apache.hadoop#hadoop-client-runtime;3.3.4 in central
        found org.apache.hadoop#hadoop-client-api;3.3.4 in central
        found commons-logging#commons-logging;1.1.3 in central
```

• Gold Layer – Dimensional Modelling

The Gold layer reads from the Silver Delta table and produces dimension tables and a fact table following a Star Schema design.

These tables are optimized for BI tool consumption.

1) **Dimension** : Region (dim_region)

Maps each sensor region to its geographic type and inherent disaster risk level :

Region	Region Type & Disaster Risk
COASTAL	Coastal Zone — HIGH disaster risk
HILLSIDE	Elevated Terrain — MEDIUM disaster risk
FOREST	Forested Area — MEDIUM disaster risk
URBAN	Urban Center — LOW disaster risk
RIVERBANK	Flood Plain — HIGH disaster risk
DESERT	Arid Zone — LOW disaster risk

2) **Dimension** : Station (dim_station)

Maps each monitoring station to its coverage type and emergency response time :

Station IDs	Station Type & Response Time
DS100, DS200	Primary Station — 10 minute response time
DS300, DS400	Secondary Station — 25 minute response time
DS500	Remote Outpost — 45 minute response time

3) **Fact Table** : fact_disaster

The central fact table contains all enriched sensor readings with dimension keys for joining :

```
# ── Fact Table: Disaster Events ──────────────────────────────
fact_stream = silver_stream.select(
    "sensor_id",
    "station_id","region",
    "threat_index_int","alert_level",
    "disaster_type","threat_band",
    "critical_flag","night_flag",
    "hour","event_ts"
)
```

4) Implementing Gold Layer

```
Windows PowerShell (x86)
PS C:\Users\sahoo\giit> docker exec -it spark-worker /opt/spark/bin/spark-submit --conf spark.jars.ivy=/tmp/.
ivy --packages io.delta:delta-spark_2.12:3.2.0,org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 /opt/spark-ap
ps/disaster-gold.py
:: loading settings :: url = jar:file:/opt/spark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings
.xml
Ivy Default Cache set to: /tmp/.ivy/cache
The jars for the packages stored in: /tmp/.ivy/jars
io.delta#delta-spark_2.12 added as a dependency
org.apache.spark#spark-sql-kafka-0-10_2.12 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-e2ca0ca5-49e7-4d12-a2ce-d7f894a8cb82;1.0
        confs: [default]
        found io.delta#delta-spark_2.12;3.2.0 in central
        found io.delta#delta-storage;3.2.0 in central
        found org.antlr#antlr4-runtime;4.9.3 in central
        found org.apache.spark#spark-sql-kafka-0-10_2.12;3.5.1 in central
        found org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.5.1 in central
        found org.apache.kafka#kafka-clients;3.4.1 in central
        found org.lz4#lz4-java;1.8.0 in central
        found org.xerial.snappy#snappy-java;1.1.10.3 in central
        found org.slf4j#slf4j-api;2.0.7 in central
        found org.apache.hadoop#hadoop-client-runtime;3.3.4 in central
        found org.apache.hadoop#hadoop-client-api;3.3.4 in central
        found commons-logging#commons-logging;1.1.3 in central
        found com.google.code.findbugs#jsr305;3.0.0 in central
        found org.apache.commons#commons-pool2;2.11.1 in central
:: resolution report :: resolve 5122ms :: artifacts dl 1352ms
        :: modules in use:
        com.google.code.findbugs#jsr305;3.0.0 from central in [default]
```

4) Spark Master UI — Pipeline Jobs Running Successfully

The Apache Spark Master Web UI at spark://spark-master:7077 confirms the cluster is fully operational with 1 alive worker node using 2 cores and 1024 MiB of memory.

The **DisasterAlertGoldLayer** job is actively running (submitted at 09:08:28), processing enriched **Silver** data into **Gold** dimension and fact tables, while the **DisasterAlertSilverLayer** job has already completed successfully after 11 minutes of execution.

Spark 3.5.1 **Spark Master at spark://spark-master:7077**

URL: spark://spark-master:7077
Alive Workers: 1
Cores in use: 2 Total, 2 Used
Memory in use: 2.0 GiB Total, 1024.0 MiB Used
Resources in use:
Applications: 1 Running, 1 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

Workers (1)

Worker Id	Address	State	Cores	Memory	Resources
worker-20260328083147-172.18.0.6-34131	172.18.0.6:34131	ALIVE	2 (2 Used)	2.0 GiB (1024.0 MiB Used)	

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260328090828-0001 (kill)	DisasterAlertGoldLayer	2	1024.0 MiB		2026/03/28 09:08:28	spark	RUNNING	41 s

Completed Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260328085220-0000	DisasterAlertSilverLayer	2	1024.0 MiB		2026/03/28 08:52:20	spark	FINISHED	11 min

- ## Spark Warehouse Directory Setup — Inside spark-worker Container

Using docker exec -it spark-worker bash, the terminal enters the spark-worker container directly to perform warehouse configuration. A Spark warehouse directory is created at **/tmp/spark-warehouse** using **mkdir -p**, and full **read/write/execute** permissions are granted recursively using **chmod -R 777** to ensure all Spark streaming jobs can write Delta Lake tables without permission errors.

This is a one-time setup step required before running the **Bronze, Silver, and Gold** layer jobs.

```
Windows PowerShell (x86)
PS C:\Users\sahoo\giit> docker exec -it spark-worker bash
spark@spark-worker:/opt/spark/work-dir$ mkdir -p /tmp/spark-warehouse
spark@spark-worker:/opt/spark/work-dir$ chmod -R 777 /tmp/spark-warehouse
spark@spark-worker:/opt/spark/work-dir$
```

- ## Spark Launching Spark SQL Shell — Delta Lake & Hive Metastore Configuration

The spark-sql shell is launched inside the spark-worker container with all necessary configurations for the pipeline. The Delta Lake package **(delta-spark_2.12:3.2.0**) is loaded, and Spark is configured to use the Delta catalog extension along with Hive as the catalog implementation connected to the Hive Metastore at **thrift://hive-metastore:9083.**

```
Windows PowerShell (x86)
spark@spark-worker:/opt/spark/work-dir$ /opt/spark/bin/spark-sql \
> --packages io.delta:delta-spark_2.12:3.2.0 \
> --conf spark.jars.ivy=/tmp/.ivy \
> --conf spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension \
> --conf spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.
DeltaCatalog \
> --conf spark.sql.catalogImplementation=hive \
> --conf spark.hadoop.hive.metastore.uris=thrift://hive-metastore:9083 \
> --conf spark.sql.warehouse.dir=/tmp/spark-warehouse
```

The warehouse directory is pointed to /tmp/spark-warehouse to ensure all Delta Lake tables are stored in the correct location accessible across the cluster.

- ## Creating & Selecting the Disaster Database in Spark SQL

Inside the Spark SQL shell, a new database named DISASTER is created using the CREATE DATABASE command, completing in 9.9 seconds as it registers the database in the Hive Metastore.

The USE DISASTER command then switches the active context to this database, confirmed by the prompt changing from **spark-sql (default)> to spark-sql (disaster)>.**

```
Windows PowerShell (x86)
spark-sql (default)> CREATE DATABASE DISASTER;
Time taken: 9.902 seconds
spark-sql (default)> USE DISASTER;
Time taken: 0.151 seconds
spark-sql (disaster)>
```

All three Gold layer Delta tables are created inside the disaster database using **CREATE TABLE**

USING DELTA with their respective warehouse storage locations. The HiveExternalCatalog warnings are expected and non-critical — they simply indicate that Delta Lake tables are being persisted into the Hive Metastore in Spark SQL format rather than native Hive SerDe format.

All three tables — fact_disaster, dim_region, and dim_station — are created successfully and are ready to receive streaming data from the Gold layer Spark job.

<u>fact_disaster Delta Table</u> : The central fact table fact_disaster is created in the disaster database using Delta Lake format, with its storage location set to **/opt/spark/warehouse/fact_disaster**.

```
Windows PowerShell (x86)
spark-sql (disaster)> CREATE TABLE fact_disaster
                    > USING DELTA
                    > LOCATION '/opt/spark/warehouse/fact_disaster';
26/03/30 16:34:42 WARN HiveExternalCatalog: Couldn't find corresponding
Hive SerDe for data source provider delta. Persisting data source table
`spark_catalog`.`disaster`.`fact_disaster` into Hive metastore in Spark
SQL specific format, which is NOT compatible with Hive.
26/03/30 16:34:43 WARN SessionState: METASTORE_FILTER_HOOK will be ignor
ed, since hive.security.authorization.manager is set to instance of Hive
AuthorizerFactory.
Time taken: 20.767 seconds
spark-sql (disaster)>
```

dim_region Delta Table : The dim_region dimension table is created using Delta Lake format and registered in the Hive Metastore at /opt/spark/warehouse/dim_region. It completes in 2.3 seconds and will store region type and disaster risk mappings produced by the Gold layer job.

```
Windows PowerShell (x86)
spark-sql (disaster)> CREATE TABLE dim_region
                    > USING DELTA
                    > LOCATION '/opt/spark/warehouse/dim_region';
26/03/30 16:38:01 WARN HiveExternalCatalog: Couldn't find corresponding
Hive SerDe for data source provider delta. Persisting data source table
`spark_catalog`.`disaster`.`dim_region` into Hive metastore in Spark SQL
 specific format, which is NOT compatible with Hive.
Time taken: 2.372 seconds
spark-sql (disaster)>
```

dim_station Delta Table : The dim_station dimension table is created at /opt/spark/warehouse/dim_station to store station type and response time mappings. It completes in 3.2 seconds and is immediately ready to receive streaming data from the Gold layer Spark job.

```
Windows PowerShell (x86)
spark-sql (disaster)> CREATE TABLE dim_station
                    > USING DELTA
                    > LOCATION '/opt/spark/warehouse/dim_station';
26/03/30 16:40:13 WARN HiveExternalCatalog: Couldn't find corresponding
Hive SerDe for data source provider delta. Persisting data source table
`spark_catalog`.`disaster`.`dim_station` into Hive metastore in Spark SQ
L specific format, which is NOT compatible with Hive.
Time taken: 3.282 seconds
spark-sql (disaster)>
```

```
Windows PowerShell (x86)
spark-sql (disaster)> SELECT * FROM dim_station;
DS400   Secondary Station       25
DS300   Secondary Station       25
DS200   Primary Station 10
DS100   Primary Station 10
DS500   Remote Outpost  45
Time taken: 5.903 seconds, Fetched 5 row(s)
spark-sql (disaster)> SELECT * FROM dim_station;
DS400   Secondary Station       25
DS300   Secondary Station       25
DS200   Primary Station 10
DS100   Primary Station 10
DS500   Remote Outpost  45
Time taken: 1.695 seconds, Fetched 5 row(s)
spark-sql (disaster)> SELECT * FROM dim_region;
FOREST  Forested Area   MEDIUM
RIVERBANK       Flood Plain     HIGH
HILLSIDE        Elevated Terrain        MEDIUM
COASTAL Coastal Zone    HIGH
URBAN   Urban Center    LOW
DESERT  Arid Zone       LOW
Time taken: 6.31 seconds, Fetched 6 row(s)
spark-sql (disaster)> SELECT COUNT(*) FROM fact_disaster;
54
Time taken: 1.108 seconds, Fetched 1 row(s)
spark-sql (disaster)>
```

• BI Views Creation – SQL Layer for Power BI Consumption

The Gold layer tables **(fact_disaster, dim_region, dim_station)** are stored as Delta Lake tables inside the Spark warehouse. While these tables hold all the processed and enriched data, they cannot be directly consumed by Power BI in a clean and reliable manner due to data type .

To bridge this gap, three dedicated BI views are created on top of the **Gold** layer tables using explicit **CAST** statements to ensure every column has the exact correct data type Power BI expects — **STRING, INT, DOUBLE, TIMESTAMP, and DATE**.

bi_fact_disaster : The bi_fact_disaster view is created on top of the fact_disaster Delta table, casting all columns to their correct data types for Power BI consumption .

Including threat_index_int as DOUBLE, event_ts as TIMESTAMP, and critical_flag and night_flag as INT.

```
Windows PowerShell (x86)
spark-sql (disaster)> CREATE VIEW bi_fact_disaster AS
                    > SELECT
                    >     CAST(sensor_id          AS STRING)    AS sensor_id,
                    >     CAST(station_id         AS STRING)    AS station_id,
                    >     CAST(region             AS STRING)    AS region,
                    >     CAST(threat_index_int   AS DOUBLE)    AS threat_index,
                    >     CAST(alert_level        AS INT)       AS alert_level,
                    >     CAST(disaster_type      AS STRING)    AS disaster_type,
                    >     CAST(threat_band        AS STRING)    AS threat_band,
                    >     CAST(critical_flag      AS INT)       AS critical_flag,
                    >     CAST(night_flag         AS INT)       AS night_flag,
                    >     CAST(event_ts           AS TIMESTAMP) AS event_time,
                    >     CAST(date               AS DATE)      AS event_date,
                    >     CAST(hour               AS INT)       AS event_hour
                    > FROM fact_disaster;
Time taken: 6.566 seconds
spark-sql (disaster)>
```

bi_dim_region & bi_dim_station : bi_dim_region exposes region, region_type, and disaster_risk from dim_region, while bi_dim_station exposes station_id, station_type, and response_time_mins from dim_station.

Both views complete almost instantly (1.2s and 0.5s respectively) and provide the lookup dimension data needed for joining with the fact view in Power BI dashboards

```
Windows PowerShell (x86)
spark-sql (disaster)> CREATE OR REPLACE VIEW bi_dim_region AS
                    > SELECT
                    >     CAST(region         AS STRING) AS region,
                    >     CAST(region_type    AS STRING) AS region_type,
                    >     CAST(disaster_risk  AS STRING) AS disaster_risk
                    > FROM dim_region;
Time taken: 1.251 seconds
spark-sql (disaster)> CREATE OR REPLACE VIEW bi_dim_station AS
                    > SELECT
                    >     CAST(station_id          AS STRING) AS station_id,
                    >     CAST(station_type        AS STRING) AS station_type,
                    >     CAST(response_time_mins  AS INT)    AS response_time_mins
                    > FROM dim_station;
Time taken: 0.468 seconds
spark-sql (disaster)>
```

- ## Starting Spark Thrift Server — Enabling Power BI Connectivity

The Spark Thrift Server (HiveThriftServer2) is started inside the spark-worker container with full Delta Lake and Hive Metastore configurations, connecting to the Spark master at spark://spark-master:7077 and the Hive Metastore at thrift://hive-metastore:9083.

Once started, the Thrift Server listens on port 10000 and acts as the JDBC/ODBC bridge that allows Power BI to connect to the disaster database and query all three BI views in real time via DirectQuery mode.

```
Windows PowerShell (x86)

spark@spark-worker:/opt/spark/work-dir$ cd /opt/spark/jars
spark@spark-worker:/opt/spark/jars$ wget https://repo1.maven.org/maven2/io/d
elta/delta-spark_2.12/3.2.0/delta-spark_2.12-3.2.0.jar
--2026-03-30 17:05:17--  https://repo1.maven.org/maven2/io/delta/delta-spark
_2.12/3.2.0/delta-spark_2.12-3.2.0.jar
Resolving repo1.maven.org (repo1.maven.org)...
104.18.18.12, 104.18.19.12, 104.18.18.12, ...
Connecting to repo1.maven.org (repo1.maven.org)|104.18.18.12|:443... connect
ed.
HTTP request sent, awaiting response... 200 OK
Length: 6111817 (5.8M) [application/java-archive]
Saving to: 'delta-spark_2.12-3.2.0.jar'

       delta-spark   0%[                 ]   8.00K  --.-KB/s               ^
   delta-spark_2.1   0%[                 ]  17.36K  3.78KB/s    eta 48m 40s^
delta-spark_2.12-3 100%[================>]   5.83M   605KB/s    in 17s

2026-03-30 17:05:37 (355 KB/s) - 'delta-spark_2.12-3.2.0.jar' saved [6111817
/6111817]
```

```
Windows PowerShell (x86)

spark@spark-worker:/opt/spark/jars$ /opt/spark/sbin/start-thriftserver.sh \
> --master spark://spark-master:7077 \
> --conf spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension \
> --conf spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.
DeltaCatalog \
> --conf spark.sql.catalogImplementation=hive \
> --conf spark.hadoop.hive.metastore.uris=thrift://hive-metastore:9083 \
> --conf spark.sql.warehouse.dir=/opt/spark/warehouse
starting org.apache.spark.sql.hive.thriftserver.HiveThriftServer2, logging t
o /opt/spark/logs/spark--org.apache.spark.sql.hive.thriftserver.HiveThriftSe
rver2-1-spark-worker.out
spark@spark-worker:/opt/spark/jars$
```

• Visualizing in Power BI

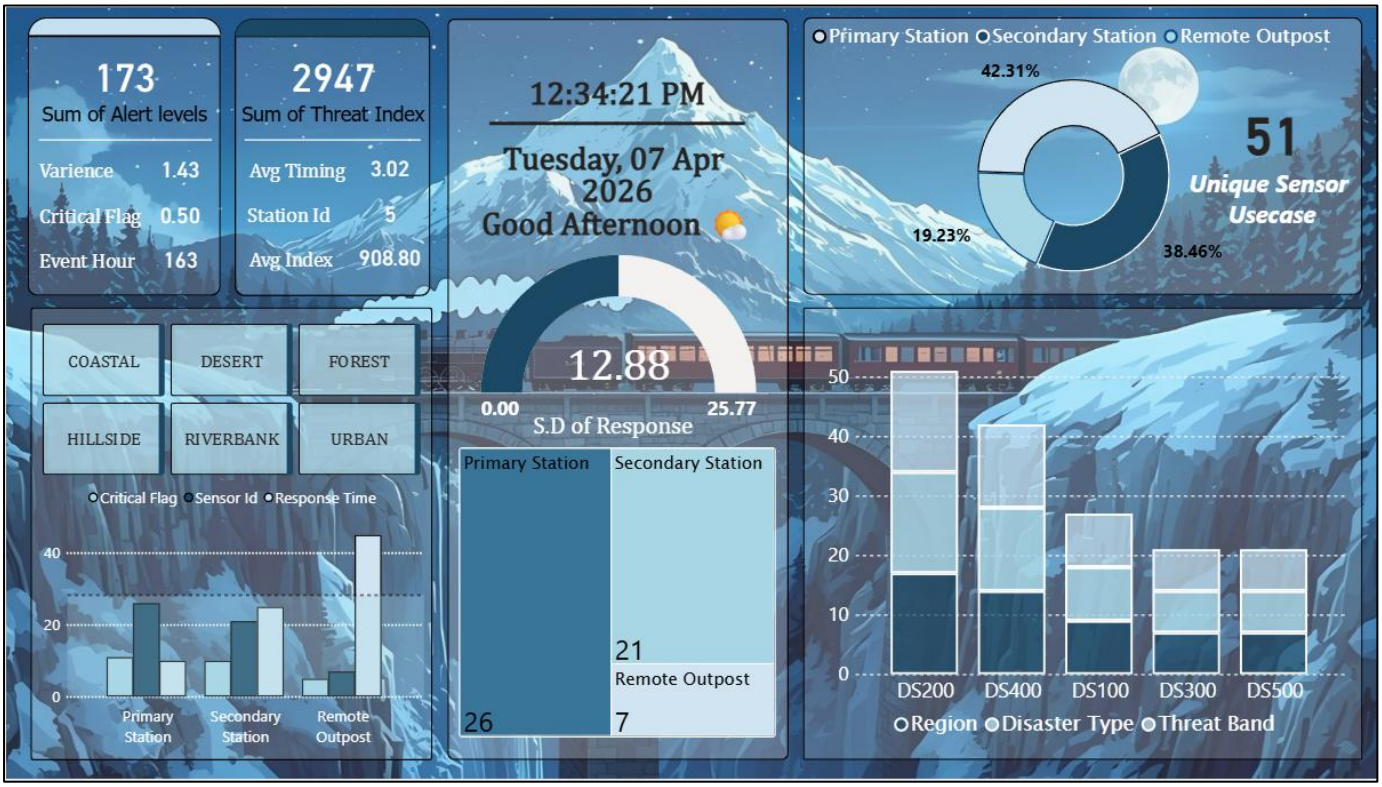
Power BI connects to Spark via the Thrift Server **(port 10000)** for live **DirectQuery** analysis:

- Dashboards: Disaster Overview, Regional Risk, Station Performance, Disaster Type Analysis
- Features: Region slicers, KPI cards, gauge charts, tree maps, trend charts
- Custom tooltips for hover-based quick insights
- Connected to disaster database via Apache Spark connector at localhost:10000

1) BI setup :

Setting	Value
Connector	Apache Spark (Get Data → Spark)
Server	localhost:10000
Protocol	Standard
Database	disaster
Mode	Direct Query
Thrift Server	Started on spark-worker container

2) Page Name : Global Monitoring Dashboard



*** Power BI desktop design workspace**

Key Insights :

KPI Cards : Displays core metrics like alert levels, threat index, variance, Avg timing, and critical flag

Real-Time Clock Panel : Shows current time, date, and contextual greeting

Threat Distribution (Donut Chart) : Proportion of alerts from Primary, Secondary, and Remote stations

Unique Sensor KPI : Total number of unique sensors use cases

Region Filter : Toggle between Coastal, Desert, Forest, Hillside, Riverbank, Urban

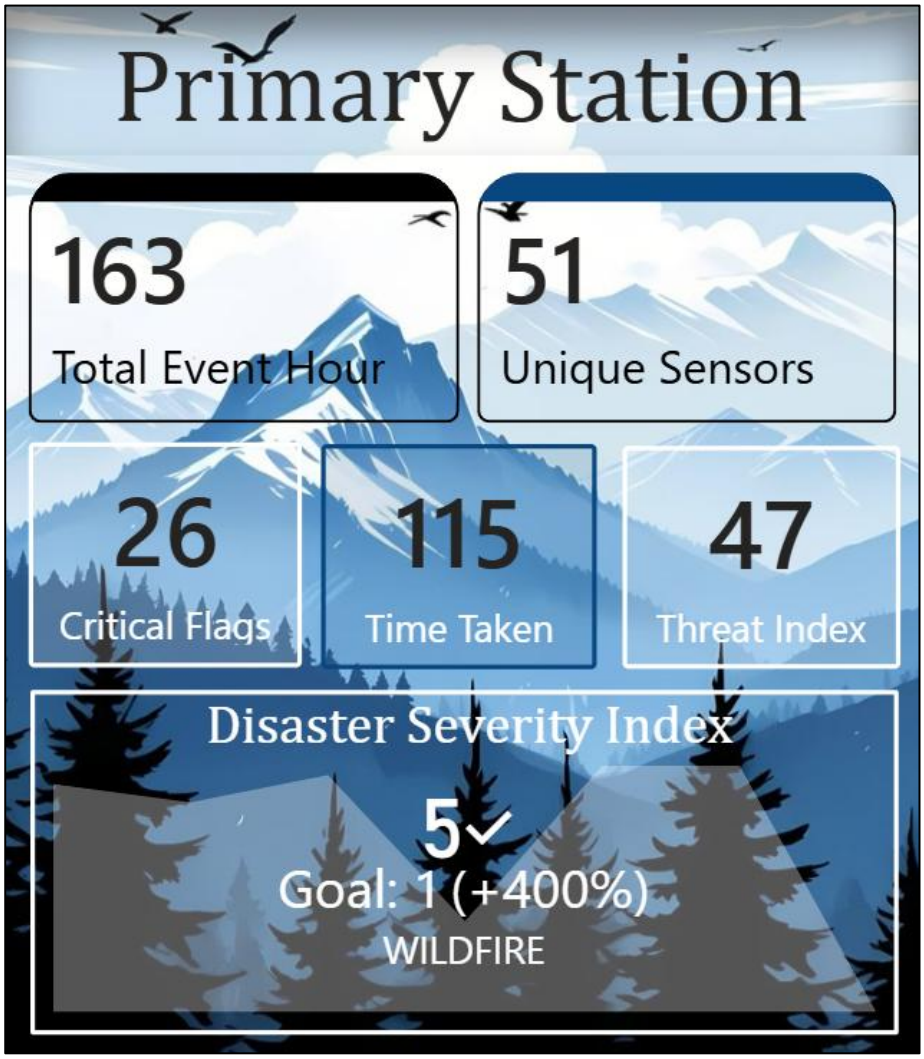
Station Comparison Chart : Compares critical flag, sensor count, and response time across stations

Response Time Gauge : Average response time with min–max scale

Station Contribution Panel : Count of events by Primary, Secondary, and Remote stations

Threat Band Analysis Chart : Comparison across datasets (DS100–DS500) by region/disaster/threat band.

2) Page Name : Tooltip [station_type]



Key Insights :

Dynamic Title : Displays selected station (e.g., Primary Station) based on map interaction

Total Event Hour : Shows total event duration/count for the selected region

Unique Sensors : Number of distinct sensors active in that location

Critical Flags : Count of high-priority or critical alerts

Time Taken : Total/average response time for events

Threat Index : Aggregated threat severity score

Disaster Severity Index : Overall severity indicator with benchmark comparison (goal vs actual)

Disaster Type Highlight : Displays dominant disaster type (e.g., Wildfire) for selected region

Contextual Filtering : Updates dynamically based on map hover/selection for region-level insights

The tooltip is dynamically linked to the **Treemap visual**, capturing the selected region or segment on hover. As users interact with the treemap, the tooltip updates in real time to reflect station-specific KPIs and disaster insights for that particular region, enabling deeper contextual analysis without navigating away.

- ## Project Summary

1) Python – Real-Time Data Simulation & Streaming

Python is used to generate and stream real-time disaster events into the pipeline.
Simulated sensor data using **Faker** (regions, stations, disaster types)
Generated **70% clean** and **30% dirty** data to mimic real-world scenarios
Introduced anomalies like nulls, duplicates, extreme values, schema drift, and corrupt JSON.
Managed timestamps with **time zone support (pytz)**
Streamed continuous events to Kafka using Kafka Producer

---

2) Apache Kafka – Real-Time Data Streaming

Kafka is used as the backbone for real-time data ingestion.
Created topic **disaster-alert-topic** for streaming events
Enabled high-throughput, fault-tolerant data streaming
Integrated producer-consumer architecture for continuous data flow
Served as entry point for Spark Structured Streaming pipeline

---

3) Apache Spark – Data Processing (Medallion Architecture)

Spark Structured Streaming is used to process data across multiple layers.
Bronze Layer : Ingested raw Kafka data into Delta Lake (schema flexible)
Silver Layer : Applied data cleaning, validation, deduplication, and type casting
Performed feature engineering **(threat_band, critical_flag, night_flag)**
Gold Layer : Built optimized analytical tables for BI consumption
Enabled distributed processing for scalability and performance

---

4) Delta Lake – Storage Layer (Lakehouse Architecture)

Delta Lake is used for reliable and scalable data storage.
Stored data across **Bronze, Silver, and Gold** layers

Provided ACID transactions and schema enforcement
Enabled efficient streaming + batch processing
Ensured data consistency and fault tolerance

---

5) Hive Metastore – Metadata Management

Hive Metastore is used to manage table schemas and metadata.
Registered Delta tables (fact_disaster, dim_region, dim_station)
Enabled structured access to data via Spark SQL
Acted as centralized metadata repository
Supported integration with BI tools via Thrift Server

---

6) SQL (Spark SQL) – Data Modeling & BI Views

SQL is used for structuring and preparing data for analytics.
• Designed Star Schema **(fact + dimension tables)**
• Created BI views **(bi_fact_disaster, bi_dim_region, bi_dim_station)**
• Applied CAST operations for Power BI compatibility
• Used Spark SQL for querying and transformations
• Enabled real-time querying through Thrift Server

---

7) Power BI – Visualization & Dashboarding

Power BI is used to create interactive dashboards for insights.
Global Monitoring Dashboard

Implemented features such as :
KPI cards, gauge charts, donut charts, treemaps
Slicers and filters for dynamic analysis
Trend and comparison visuals

Added advanced capabilities :
Custom tooltips linked with treemap for drill-down insights
Real-time monitoring using DirectQuery
Interactive visuals for decision-making

---

8. Spark Thrift Server – BI Connectivity Layer

Acts as a bridge between Spark and Power BI.
Enabled JDBC/ODBC connectivity on port 10000
Allowed Power BI to query Spark tables in real time
Supported DirectQuery mode for live dashboards